```
!python --version
```

```
Python 3.12.12
```

```
# !pip install arviz pymc pytensor
```

```
import arviz as az
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pymc as pm
from pymc import GARCH11
from pytensor import shared
import pytensor
pytensor.config.mode = 'NUMBA'
```

```
# Download the dataset
m5_data = pd.read_csv("M5. goog_eur_10.csv")

# Convert date variable to date format
m5_data["Date2"] = pd.to_datetime(m5_data["Date"], format="%m/%d/%Y")
goog = m5_data.loc[:, ["Date2", "GOOGLE"]].set_index("Date2")
goog["GOOGLE_R"] = np.log(goog.GOOGLE).diff().dropna()
```

1. Bayesian Statistics

1.1 What is Bayesian Statistics?

Bayesian statistics uses Bayes' theorem as the foundation to apply probabilities to data analysis. From the method we have been learning so far, we assume parameters like mean, variance, and standard deviations are unknown but fixed parameters. This method is called **frequentist statistics.** Maximum likelihood estimation belongs to frequentist statistics. On the other hand, in Bayesian statistics, mean, variance and standard deviations are random variables with probability distributions. This is the main difference between frequentist statistics and Bayesian statistics. To run Bayesian statistics, a researcher has to either use experience or other techniques to assume a parameter's probability distribution before running the analysis. We call these probability distributions **prior probability distributions** or **priors**. We then combine the information from data to derive **posterior probability distributions for parameters** or **posteriors.** The required readings in this lesson will provide definitions and applications of these concepts in more detail. It is good that now the parameters are not unknown fixed values but rather random variables. This change can make analysis more flexible. However, this method sometimes is criticized for subjective parameter probability distribution choice by the researcher before running the data analysis. There is a lot of debate about advantages and disadvantages of frequentist statistics vs. Bayesian statistics. For students who are interested in knowing more detail, there is a lot of material online.

1.2 Basics of Bayesian Updating, Prior Probability Distributions, Posterior Probability Distributions, and Likelihood Function

Before continuing this lesson, please read the required readings for this lesson. The required readings will cover the concept of Bayesian updating, prior probability distributions (priors), posterior probability distributions (posteriors), and likelihood functions in detail. In the following sections of this lesson, we will use these concepts extensively so it is important that you are familiar with these concepts before proceeding to the next section.

1.3 Sampling from Posterior Probability Distribution

We learned in the last section how to derive posterior probability distributions for parameters we are interested in analyzing. We know a posterior distribution is the multiplication of the likelihood function and the prior distribution. Oftentimes, a posterior distribution formula is too complicated to manipulate or is in high dimension. Hence, we have to conduct sampling to obtain a series of sample data points from the posterior distribution to investigate its marginal distribution density, mean, and other parameters.

The most popular sampling method is the **Markov-Chain-Monte-Carlo (MCMC)** method. Monte Carlo is a random sampling method when the samples are independent. Markov-Chains address random samples that are autocorrelated to each other. Hence, MCMC method is a random sampling method for autocorrelated samples. There are several sampling algorithms under the MCMC method. The two most popular ones are the **Metropolis-Hastings algorithm** and the **Gibbs Sampling algorithm**. For those who are interested in the details of these algorithms, there are plenty of materials to explore online. For the following sections, the software we are using to conduct Bayesian estimation on GARCH models uses the Metropolis-Hastings algorithm.

One final note about sampling before we move on to Bayesian estimation of GARCH models: The MCMC method can be sensitive to the initial values used to start sampling. Therefore, sometimes, we would use the result from maximum likelihood estimation as the starting values for Bayesian estimation. We will demonstrate this point when we show how to use Bayesian estimation in a later section.

2. Bayesian Estimation on GARCH Model

In this section, we are going to go over the steps to use Bayesian method to fit a GARCH model. Specifically, we will focus on a GARCH model with Student's t innovations (errors). From the last lesson, we see that one common feature of financial asset returns is their heavy tail distribution. Hence, it is reasonable to use Student's t-distribution to model asset returns.

Ardia and Hoogerheide (2010) demonstrated the steps using Bayesian estimation to fit a GARCH model with Student's t innovations. They wrote an R package (bayesGARCH) to automate the sampling process by using the Metropolis-Hastings algorithm. In the next section, we will use our Google stock return as an example to show the process. Before diving into the example, let's go over the theory for GARCH model with Student's t innovations first.

$$r_t = \epsilon_t \sqrt{\frac{\nu - 2}{\nu} \omega_t \sigma_t^2}$$

where

> $t = 1, \cdots, T$
> $\epsilon_t \sim \text{iid normal}(0, 1)$
> $\omega_t \sim \text{iid inverse gamma}(\frac{\nu}{2}, \frac{\nu}{2})$
> $\sigma_t^2 = \alpha_0 + \alpha_1 r_{t-1}^2 + \beta \sigma_{t-1}^2$ where $\alpha_0 > 0,\ \alpha_1, \beta \geq 0$ and $\nu > 2$.

The restrictions above will ensure that the variance will be positive and finite. However, they do not ensure stationarity. We know from the last lesson that $\alpha_1 + \beta < 1$ is needed to ensure stationary. We will use the result in our application later.

Another point we would like to explain is the inclusion of ω_t. The reason we have ω_t as a prior multiplied with the variance of r_t in the model is that the marginal distribution of r_t will become a Student's t-distribution with degrees of freedom ν.

The priors for $\alpha_0, \alpha_1, \beta$ are assumed to be truncated normal distributions with the domains greater than 0 since these restrictions ensure positive and finite variance. The prior for ν is shifted exponential distribution.

The joint posterior distribution will look like:

$$p(\alpha_0, \alpha_1, \beta, \nu, \omega_t | r_t) = \frac{l(r_t | \alpha_0, \alpha_1, \beta, \nu, \omega_t)\ p(\alpha_0, \alpha_1, \beta, \nu, \omega_t)}{p(r_t)}$$

Then, MCMC will sample from the joint posterior distribution for each parameter. From the sampled data, we can generate the density for each parameter. As we mentioned in the last section, MCMC sampling can be sensitive to initial values. One method to avoid extreme values from initial values is to drop the first n numbers of sampled observations. This is called a **burn-in** period. We can indicate to the package how many initial sampled observations we would like to drop from our sample series. We will elaborate more on this topic in the next section.

In this section, we will use Google stock return to fit a GARCH model by using the Bayesian estimation method. We will use Python package: PyMC to run this model. The PyMC package has inbuilt GARCH(1,1) distribution that will make the code easier to implement. We should bear in mind that the inbuilt GARCH distribution is with Normal innovations. However, from the Ardia and Hoogerheide (2006) we can learn that the priors for Bayesian GARCH(1,1) with Normal innovations remain the same. Hence, we provide this example for demonstration purposes.

We will use MCMC to generate two data sample series with 3000 samples in each series. In MCMC, a sample series is called a **chain**. Why do we need to sample two data series? We generate two chains because we need to use them to check if the sample series are close enough to the posterior that we want to analyze. This is called **convergence diagnostics** for MCMC sampling. There are several methods to conduct convergence diagnostics. We will use the Gelman-Rubin Diagnostic. The diagnostic provides a "Scale Reduction Factor" to indicate if several series converge. Usually, a value below 1.2 is okay. Since this diagnostic requires at least two chains, we will create two chains from MCMC. Figure 1 shows the MCMC sampling result for the joint posterior distribution for Google stock return GARCH model.

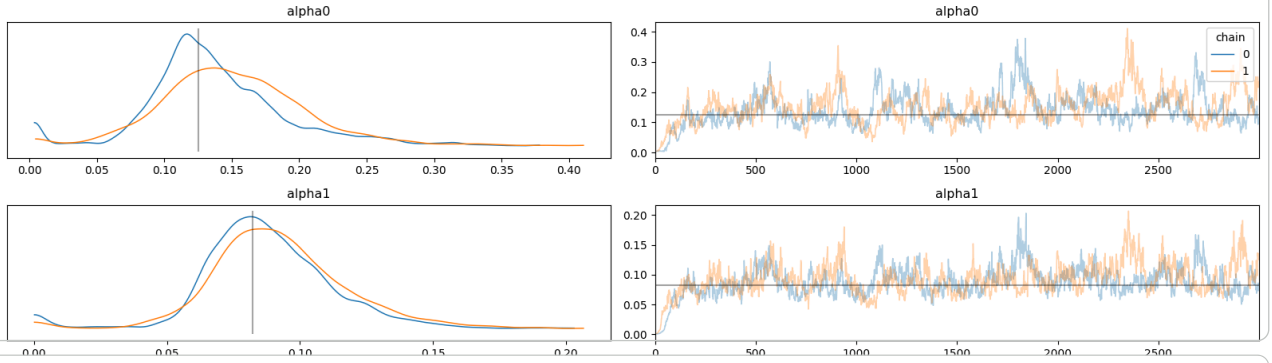
Figure 1: First Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

```
# First Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

# starting parameters = blank canvas model(0.000001, 0.000001, 0.000001)
alpha_mu = shared(np.array([0.000001, 0.000001], dtype=np.float64))
alpha_sigma = shared(np.array([[1000.0, 0.0], [0.0, 1000.0]], dtype=np.float64))

beta_mu = shared(np.array(0.000001, dtype=np.float64))
beta_sigma = shared(np.array(1000.0, dtype=np.float64))
```

```
# initial volatility
ivolatility = shared(np.array(0.000001, dtype=np.float64))
ivolatility_vol = shared(np.array(10.0, dtype=np.float64))

# construct MCMC model
mcmc0 = pm.Model()

with mcmc0:
    # create multivariate normal: p(α) ∝ ϕN2(α|μα,Σα) 1 {α ∈ R^2+}
    mvn = pm.MvNormal("mvNormal", mu=alpha_mu, cov=alpha_sigma, shape=2)

    # restrict the two coefficients to be greater than zero
    alp0 = pm.Deterministic("alpha0", pm.math.switch(mvn[0] > 0, mvn[0], -np.inf))
    alp1 = pm.Deterministic("alpha1", pm.math.switch(mvn[1] > 0, mvn[1], -np.inf))

    # create univariate truncated normal: p(β) ∝ ϕN1(β|μβ,Σβ) 1{β ∈ R+}
    nTruncated = pm.TruncatedNormal("beta", mu=beta_mu, sigma=beta_sigma, lower=0)
    volTruncated = pm.TruncatedNormal(
        "volatility", mu=ivolatility, sigma=ivolatility_vol, lower=0
    )

    # likelihood
    likelihood = GARCH11(
        "GARCH",
        omega=alp0,
        alpha_1=alp1,
        beta_1=nTruncated,
        initial_vol=volTruncated,
        observed=goog.GOOGLE_R.dropna() * 100,
    )
```

```
# Plot first round MCMC model posteriors
with mcmc0:
    step_mcmc0 = pm.Slice()
    trace_mcmc0 = pm.sample(
        3000,
        cores=2,
        step=step_mcmc0,
        tune=0,  # tune=0 => no burn-in
        return_inferencedata=True,
        random_seed=12345,
    )
    az.plot_trace(
        trace_mcmc0,
        var_names=["alpha0", "alpha1", "beta"],
        # plot vertical lines at GARCH(1,1) MLE param results
        lines=[
            ("alpha0", {}, [0.124993]),
            ("alpha1", {}, [0.082160]),
            ("beta", {}, [0.867127]),
        ],
        compact=False,
        legend=True,
        figsize=(16, 7),
    )
    plt.tight_layout()
    plt.show()
```

```
Progress                                Draws   Tuning   Steps out   Steps in   Sampling Speed    Elapsed   Remaining
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  3000    False    0           18         18.89 draws/s     0:02:38   0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  3000    False    0           10         18.82 draws/s     0:02:39   0:00:00

ERROR:pymc.stats.convergence:The effective sample size per chain is smaller than 100 for some parameters.  A higher number i
```




```
# First Round MCMC Convergence Test Result and Parameter Estimations
az.summary(trace_mcmc0, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
alpha0	0.145094	0.053294	0.059351	0.262754	0.007721	0.004627	47.599936	51.832998	1.068832
alpha1	0.089880	0.025170	0.048496	0.141094	0.003876	0.002370	47.796746	48.588985	1.066640
beta	0.852806	0.042358	0.757265	0.916286	0.006640	0.004178	44.814853	48.402712	1.074747

Last column in Figure 2 for *r_hat* represents the Gelman-Rubin statistic. It tests for lack of convergence by comparing the variance between multiple chains to the variance within each chain. If convergence has been achieved, the between-chain and within-chain variances should be identical. If *r_hat* < 1.2 for all model parameters, we can have some confidence that convergence has been reached.

Let's look at the estimations for the parameters. Figure 2 shows the estimated parameters from the first round MCMC sampling result. Let's focus on the means of the coefficients. We see *alpha0*, *alpha1* and *beta* all look reasonable within our restrictions for stationary and positive and finite variance. But so far the model represents so called 'blank canvas' model i.e. we have not accounted for any meaningful starting values. In this initial model the starting values are all near zero for all *alpha0*, *alpha1* and *beta* priors.

We discussed in the previous section that the MCMC can be sensitive to starting value. If starting value is way off, it can impact the final result of MCMC. In this case, we can assign starting values to parameters before the MCMC starts sampling. In that way, we restrain MCMC from going wild when sampling. So where can we find initial values for parameters? In the previous lesson, we estimated the parameters using maximum likelihood. We can use these estimates as starting values for MCMC. Otherwise, this step is a trial-and-error process. You might need to run MCMC a number of times for MCMC to converge to a solution and find reasonable coefficient estimates.

For the next round of MCMC, we adjust starting values for parameters in the code and rerun MCMC. The following figures show the final results of the second run MCMC sampling.

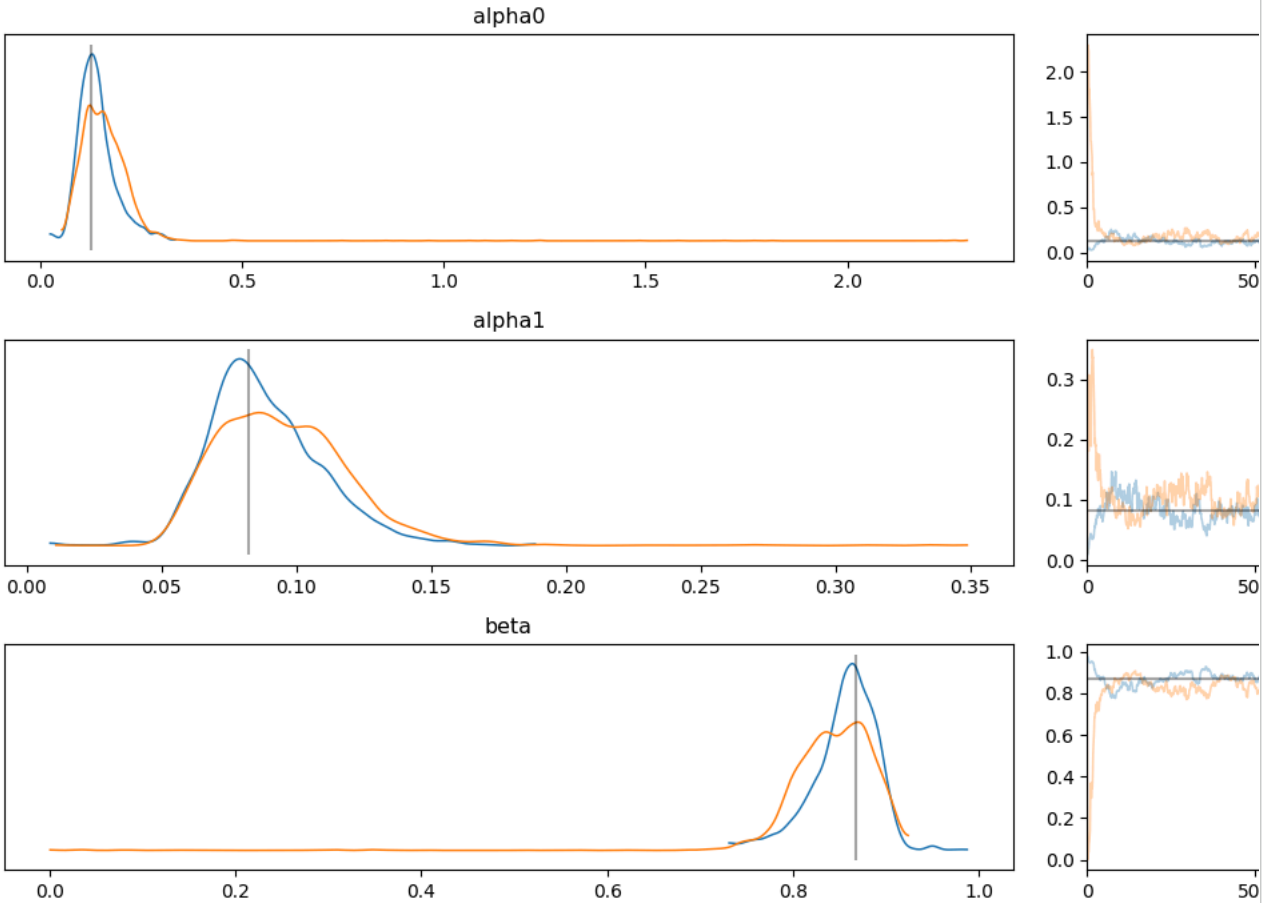
Figure 3: Second Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

```
# Second Round MCMC Sampling Result: Parameter Trace Plots and Marginal Density Plots

# starting parameters = MLE(0.124993, 0.082160, 0.867127)
alpha_mu = shared(np.array([0.124993, 0.082160], dtype=np.float64))
alpha_sigma = shared(np.array([[1000.0, 0.0], [0.0, 1000.0]], dtype=np.float64))

beta_mu = shared(np.array(0.867127, dtype=np.float64))
beta_sigma = shared(np.array(1000.0, dtype=np.float64))

# initial volatility
ivolatility = shared(
    np.array(1.63865, dtype=np.float64)
)  # np.std(goog["GOOGLE_R"].dropna()*100)
ivolatility_vol = shared(np.array(10.0, dtype=np.float64))

# construct MCMC model
mcmc = pm.Model()

with mcmc:
    # create a multivariate normal: p(α) ∝ ϕN2(α|μα,Σα) 1 {α ∈ R^2+}
    mvn = pm.MvNormal("mvNormal", mu=alpha_mu, cov=alpha_sigma, shape=2)

    # restrict the two coefficients to be greater than zero
```

```
    alp0 = pm.Deterministic("alpha0", pm.math.switch(mvn[0] > 0, mvn[0], -np.inf))
    alp1 = pm.Deterministic("alpha1", pm.math.switch(mvn[1] > 0, mvn[1], -np.inf))

    # create univariate truncated normal: p(β) ∝ ϕN1(β|μβ,Σβ) 1{β ∈ R+}
    nTruncated = pm.TruncatedNormal("beta", mu=beta_mu, sigma=beta_sigma, lower=0)
    volTruncated = pm.TruncatedNormal(
        "volatility", mu=ivolatility, sigma=ivolatility_vol, lower=0
    )

    # likelihood
    likelihood = GARCH11(
        "garch",
        omega=alp0,
        alpha_1=alp1,
        beta_1=nTruncated,
        initial_vol=volTruncated,
        observed=goog.GOOGLE_R.dropna() * 100,
    )
```

```
# Plot Second round MCMC model posteriors
with mcmc:
    step_mmc = pm.Slice()
    trace_mmc = pm.sample(
        3000,
        cores=2,
        step=step_mmc,
        tune=0,  # no burn-in
        return_inferencedata=True,
        random_seed=12345,
    )
    az.plot_trace(
        trace_mmc,
        var_names=["alpha0", "alpha1", "beta"],
        lines=[
            ("alpha0", {}, [0.124993]),
            ("alpha1", {}, [0.082160]),
            ("beta", {}, [0.867127]),
        ],
        compact=False,
        legend=True,
        figsize=(16, 7),
    )
    plt.tight_layout()
    plt.show()
```

```
Progress                                Draws   Tuning   Steps out   Steps in   Sampling Speed    Elapsed   Remaining
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  3000    False    0           11         18.86 draws/s     0:02:38   0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  3000    False    0           14         18.83 draws/s     0:02:39   0:00:00

ERROR:pymc.stats.convergence:The effective sample size per chain is smaller than 100 for some parameters.  A higher number i
```




```
Start coding or generate with AI.
```

In figure 3, we can see that after adding starting values to the code, the trace plot for all priors converges better in this round of MCMC. Let's check the Gelman-Rubin convergence test and other parameter estimations.

```
# Second Round MCMC Sampling Result: Parameter estimation
az.summary(trace_mmc, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
alpha0	0.151440	0.091908	0.067844	0.240691	0.007156	0.043919	82.938764	221.227579	1.029334
alpha1	0.092149	0.024706	0.054129	0.132585	0.002455	0.002861	92.183544	318.266329	1.024602
beta	0.848298	0.051242	0.782080	0.913977	0.004892	0.015678	75.874103	231.988360	1.030617

We can see from figure 4 that all the parameters have converged pretty well after adding starting values to the code and *r_hat* improved.

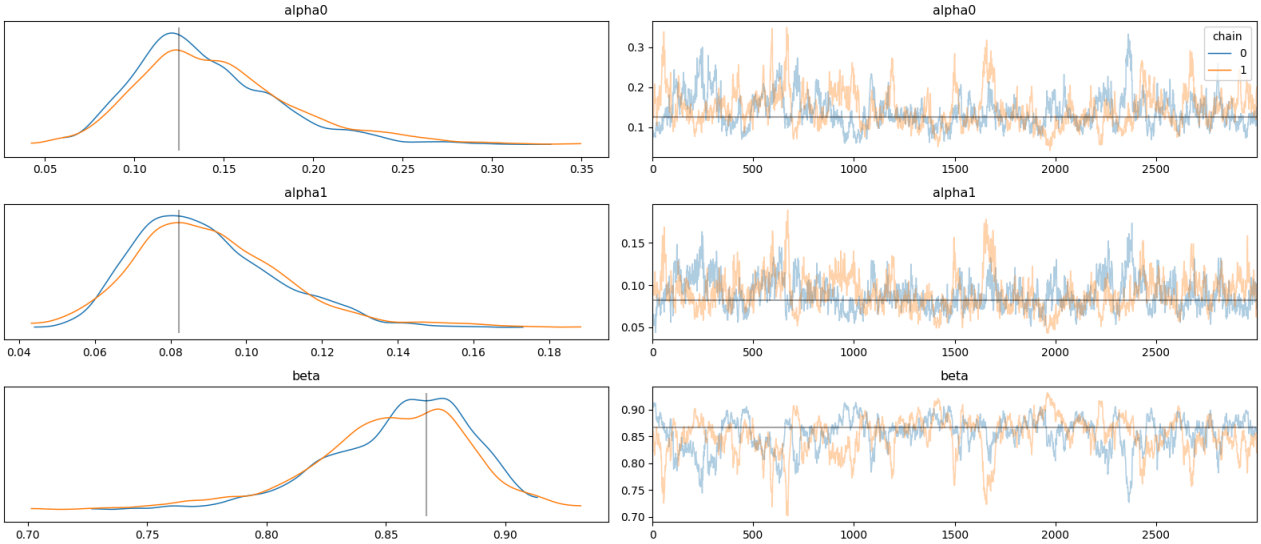
However, we can still see from figure 3 that the first 250 data points seem to be very volatile. We can treat them as the burn-in period and drop those sampled data points. At this point, we will also combine chains. Let's check out the MCMC result after these steps.

Figure 5: Second Round MCMC Sampling Result: Parameter Trace Plots and Density Plots After Adjustment

```
# Second Round MCMC Sampling Result: Parameter Trace Plots and Density Plots After Adjustment
with mcmc:
    step_mmc_a = pm.Slice()
    trace_mmc_a = pm.sample(
        3000,
        cores=2,
        step=step_mmc_a,
        tune=250,  # burn-in = 250 steps
        return_inferencedata=True,
```

```
        random_seed=12345,
    )
    az.plot_trace(
        trace_mmc_a,
        var_names=["alpha0", "alpha1", "beta"],
        lines=[
            ("alpha0", {}, [0.124993]),
            ("alpha1", {}, [0.082160]),
            ("beta", {}, [0.867127]),
        ],
        compact=False,
        legend=True,
        figsize=(16, 7),
    )
    plt.tight_layout()
    plt.show()
```

```
Progress                                  Draws   Tuning   Steps out   Steps in   Sampling Speed    Elapsed   Remaining
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━    3250    False    1           1          26.62 draws/s     0:02:02   0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━    3250    False    0           7          25.76 draws/s     0:02:06   0:00:00

ERROR:pymc.stats.convergence:The effective sample size per chain is smaller than 100 for some parameters.  A higher number i
```



From figure 5, we can see the trace plots for 3 parameters all oscillate within a narrower range compared to the trace plots in figure 1. On the left, we can see that the marginal densities of *alpha0* and *alpha1* seem to skew to the right while the marginal density for beta skew to the left. Let's check the final parameter estimates.

```
Start coding or generate with AI.
```

Figure 6: First Round MCMC Sampling Result: Parameter Estimation with Adjustment

```
# First Round MCMC Sampling Result: Parameter Estimation with Adjustment
az.summary(trace_mmc_a, round_to=7).loc[["alpha0", "alpha1", "beta"], :]
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
alpha0	0.143620	0.043415	0.074571	0.231585	0.004067	0.002347	108.231247	235.146465	1.008775
alpha1	0.090170	0.019715	0.057861	0.128158	0.001783	0.001058	119.996201	301.232062	1.003836
beta	0.852987	0.032861	0.786890	0.907703	0.003265	0.001904	97.762053	211.792705	1.007301

```
Start coding or generate with AI.
```

In figure 6, we can see that all estimated coefficients are well behaved and confined in the restrictions to meet stationarity and positive and finite variance requirements. In figure 7 below, we can compare the estimated coefficients from MLE and Bayesian estimations. We can see the estimated coefficients are similar. We achieved this comparable result partly because we used MLE estimates as our starting values for MCMC sampling. By carefully selecting starting values for MCMC, we can control the behavior of the estimated coefficients.